esprit Se former autrement HONORIS UNITED UNIVERSITIES	**Tunisian Republic** **Ministry of Higher Education and Scientific Research** **ESPRIT Private School of Engineering and Technology**	AWD Module Distributed Web Architecture

MINI-PROJECT REPORT

Distributed Web Architecture — Engineering Program

SMART CITY

Web Platform for Urban Management and Municipal Services

Functional Analysis, System Design, and Proposed Microservices Architecture

Prepared by:

Eya Hwess

Eya Abdellaoui

Sarra Guesmi

Nour Ammar

Bilel Rebhi

Academic Year: 2026/2027

1. Project overview

Smart City gives residents one website to report street problems, track municipal requests, book public facilities and take part in local consultations. Municipal staff use it to review reports, assign repairs and follow requests through to a decision.

The platform is split into microservices, each responsible for its own data and workflows. For example, the incident service manages reports and repairs, while the mobility service manages road closures and parking. ReLink AI helps residents find lost items and pets by comparing lost and found reports.

2. Problem statement

When municipal services use separate systems, residents can struggle to find the right department or check what happened after they filed a request. Staff also need a clear record of who is handling each case and what still needs to be done.

How can a city give residents one place to access municipal services while allowing each service to manage its own data and workflows?

3. Project objectives

• Provide a single web platform for important municipal and community services.

• Improve citizen participation and transparency.

• Make incident reporting and maintenance follow-up traceable.

• Improve access to traffic, parking and public-facility information.

• Monitor environmental indicators and alert on threshold breaches.

• Digitize administrative requests and appointments.

• Provide dashboards and historical indicators to support municipal decisions.

• Keep business capabilities independent through microservices and database ownership.

• Help residents recover lost items and pets by matching photos and descriptions from lost and found reports.

4. Actors

Actor	Responsibility	Typical actions
Citizen	Uses municipal services and participates in local decisions.	Manage a profile, vote or comment, report incidents, track requests and book parking or facilities.
Municipal administrator	Oversees municipal operations.	Configure services and rules, approve cases where required, review dashboards and assign resources.
Municipal agent	Processes incident reports and administrative requests.	Validate reports, request missing documents, prioritize cases and verify repairs.
Technician	Carries out field repairs.	Review assigned work orders, report progress or blockers, and record work and evidence.
Mobility operator	Manages traffic and parking information.	Update road closures, parking availability and reservation rules.
Environmental operator	Monitors environmental conditions.	Configure stations and thresholds, review readings and acknowledge alerts.
Facility manager / activity organizer	Manages facilities and community activities.	Maintain availability, review bookings, publish events and manage registrations.
Lost-and-found agent	Helps verify claims and coordinate returns.	Review ownership claims and record return outcomes.
System administrator	Maintains platform infrastructure and access.	Manage technical configuration, service discovery and operational monitoring.

5. MICROSERVICE BOUNDARIES AND FUNCTIONAL SCOPE

MICROSERVICE	BUSINESS RESPONSIBILITY	MAIN USERS
MS1 · Citizen & Civic Participation	Sign-in, sign-out and roles, plus citizen profiles, proposals, consultations, voting and comments.	All platform roles for account access · Citizens and municipal administrators for civic participation
MS2 · Urban Incident & Maintenance	Incident reports, validation, prioritization, work orders, technician interventions and resolution.	Citizen · Municipal agent · Technician · Administrator

MS3 · Smart Mobility & Traffic	Traffic information, road closures, parking availability and parking reservations.	Citizen · Mobility operator · Administrator
MS4 · EcoCity & Environmental Monitoring	Air/noise measurements, thresholds, pollution alerts and environmental trends.	Citizen · Environmental operator · Administrator
MS5 · Digital Municipal Services	Service catalog, online applications, documents, processing and appointments.	Citizen · Municipal service agent · Administrator
MS6 · Smart Places & Public Facilities	Public facilities, availability, reservations, activities and reviews.	Citizen · Facility manager · Organizer
MS7 · ReLink AI	Matches lost and found reports and supports ownership verification and returns.	Citizen · Lost-and-found agent

5.1 MS1: CITIZEN & CIVIC PARTICIPATION

Business responsibility

MS1 handles sign-in, sign-out, and role-based access for all Smart City users. It uses Keycloak to authenticate users and obtain their roles. MS1 also manages citizen profiles and civic participation. Other services use citizenId to refer to a citizen profile.

Actors

All user roles sign in and out through the user module: citizens, municipal administrators, municipal agents, technicians, mobility operators, environmental operators, facility managers, activity organizers, lost-and-found agents, and system administrators.

• **Citizen:** Manages their profile and address, submits proposals, tracks their status, comments, votes, and views their participation history.

• **Municipal administrator:** Publishes consultations, moderates proposals and comments, and reviews participation statistics.

• **System administrator:** Manages account access and role assignments.

Functionalities

• **Sign-in and sign-out:** Let users from every role start and end an authenticated session through Keycloak.

• **Roles and permissions:** Use the authenticated user’s roles to control access to platform features.

• **Citizen profiles:** Create, view, and update profiles, including neighborhood and address information.

• **Citizen proposals:** Submit suggestions such as benches, trees, or other public improvements, and track their status.

• **Public consultations:** Publish a question or project with a description, opening and closing dates, status, and voting options.

• **Voting:** Let eligible citizens select a consultation option according to the consultation's voting rules.

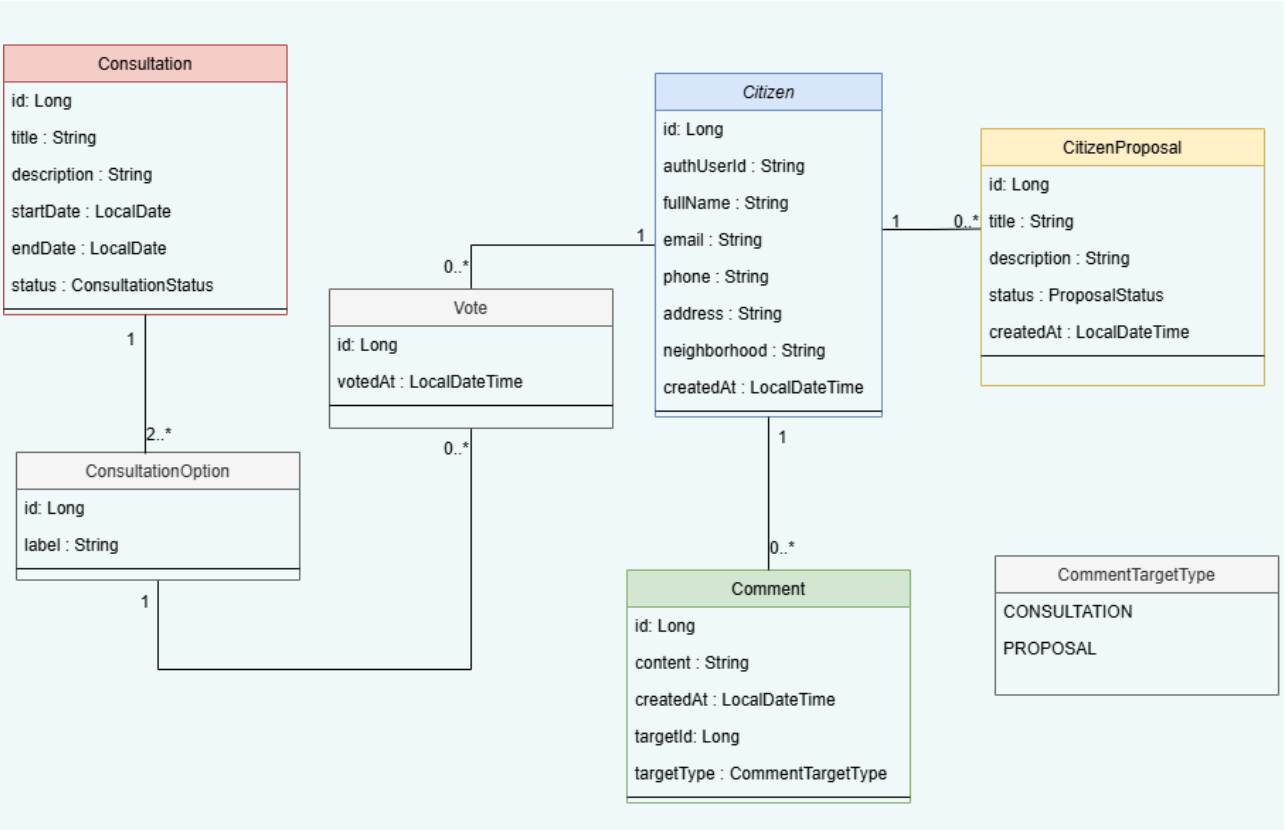
• **Comments:** Let citizens comment on proposals or consultations.

• **Participation history:** Let citizens view their proposals, votes, and comments.

• **Participation statistics:** Count proposals, votes, and active participants.

Class diagram

The class diagram covers civic participation: Citizen, CitizenProposal, Consultation, ConsultationOption, Vote, Comment, and CommentTargetType. To show the full scope of MS1, the design also needs to show how Keycloak accounts and roles relate to the citizen profile and to staff access.



5.2 MS2 — URBAN INCIDENT & MAINTENANCE MANAGEMENT

Business responsibility

This microservice manages the entire process of handling urban incidents, from the moment a citizen reports a problem until it is fixed and confirmed as resolved. It handles incident reports, validation, technician assignments, repairs, and follow-up.

Actors

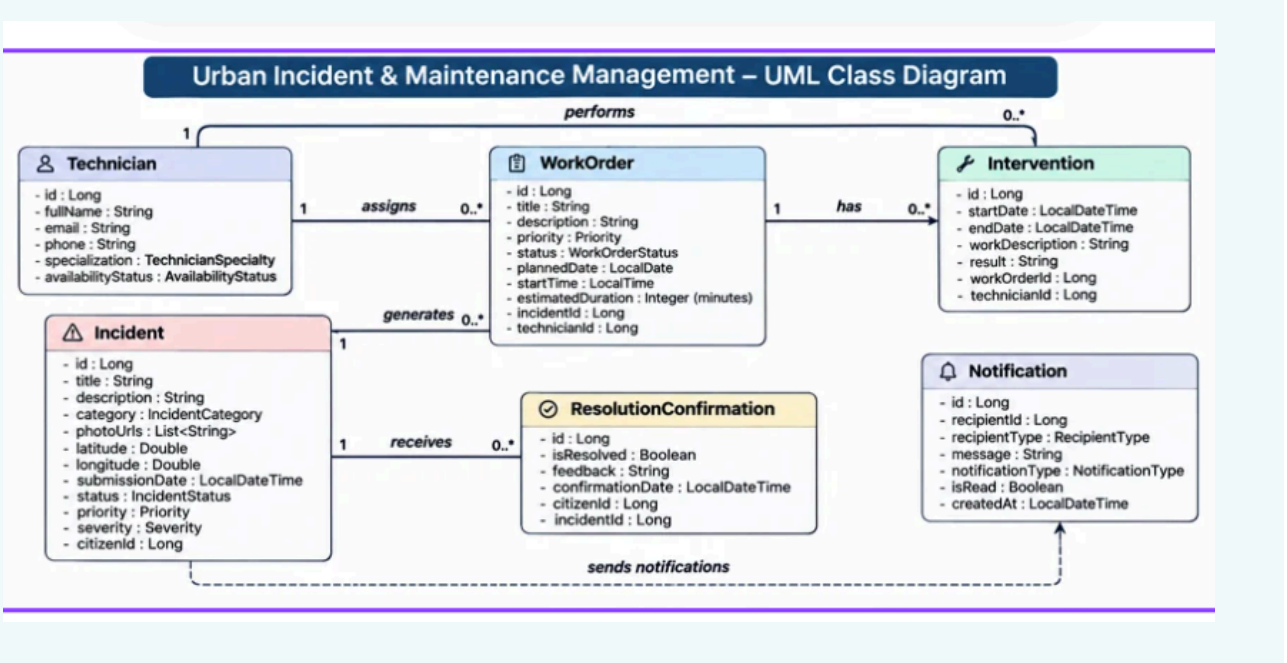
- Citizen: Reports problems in the city, adds photos and location, tracks progress, and confirms whether the problem has been solved.
- Municipal Agent: Reviews reports, checks their validity, chooses their category and priority, and verifies completed repairs.

- Technician: Receives assigned tasks, carries out repairs, and updates the progress of the work.
- Municipal Administrator: Manages technicians, organizes assignments, monitors incidents, and views statistics.

Functionalities

- Report an Incident: Citizens can report problems by providing a title, description, location, and photos.
- Categorize Incidents: Reports are organized into categories such as road damage, broken streetlights, or waste problems. AI can suggest a category, but a municipal agent makes the final decision.
- Validate Reports: Municipal agents can approve, reject, or ask for more information about an incident.
- Set Priorities: Incidents are classified as LOW, MEDIUM, HIGH, or CRITICAL. AI can suggest the urgency level, which is then confirmed by an agent.
- Create Repair Tasks: Once an incident is approved, a work order is created to organize the repair.
- Assign Technicians: The system assigns technicians based on their skills and availability.
- Schedule Repairs: Repairs are planned with a date, time, and estimated duration.
- Track Progress: Each repair has a status, such as pending, assigned, scheduled, in progress, blocked, completed, or cancelled.
- Confirm Resolution: After the repair, a municipal agent verifies the work. The citizen can also confirm that the issue is fixed or report that the problem still exists.
- Send Notifications: Citizens and technicians receive updates about incident status changes and assignments.
- View Statistics: Administrators can see the number of reported incidents, their categories and priorities, completed repairs, average resolution time, and reopened incidents.

Class Diagram



5.3 MS3 — SMART MOBILITY & TRAFFIC

Business Responsibility

This microservice helps citizens move around the city more easily by providing traffic information, managing road closures, and handling public parking.

It also receives information about traffic-related incidents from MS2, without managing the original incident reports.

Actors

• **Citizen / Driver:** Checks traffic conditions, finds available parking spaces, makes reservations, and manages bookings.

• **Mobility Operator:** Updates traffic information, manages road closures, and monitors parking availability.

• **Municipal Administrator:** Sets parking rules and monitors traffic and parking statistics.

Main Functionalities

1. **Traffic Monitoring:** Display current or simulated traffic conditions on different roads.

2. **Road Closure Management:** Manage closed or restricted roads, including the reason and closure period.

3. **Traffic Disruptions:** Inform drivers about accidents, roadworks, or other problems affecting traffic.

4. **Parking Management:** Add and manage public parking areas and their rules.

5. **Parking Availability:** Show available and occupied parking spaces.

6. **Parking Reservations:** Allow citizens to book an available parking space.

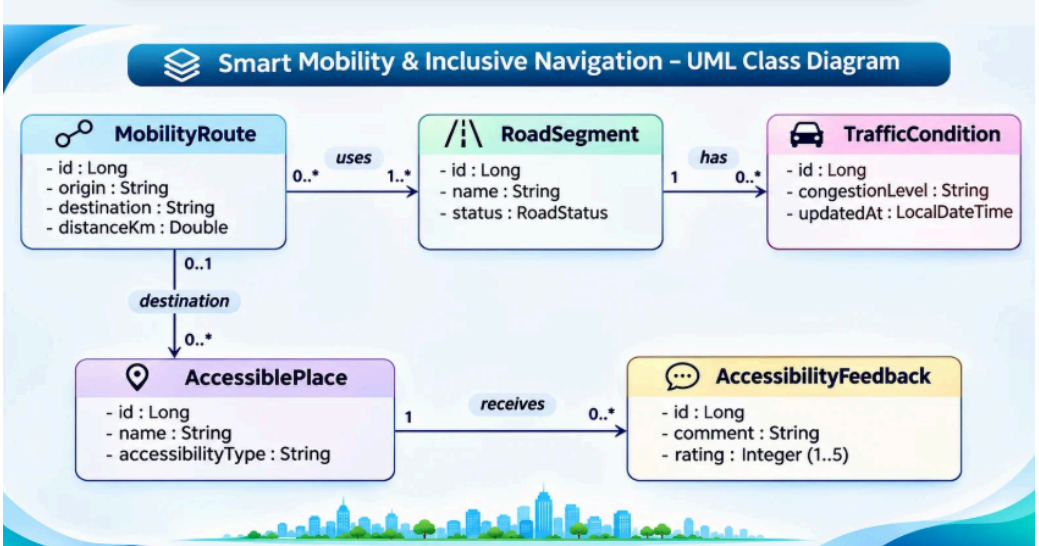
7. **Manage Reservations:** View, modify, or cancel bookings according to parking rules.

8. **Parking History:** Allow citizens to see their previous parking reservations.

9. **Alternative Routes:** Suggest other routes when roads are blocked or congested, using a routing service when available.

10. **Mobility Statistics:** Display parking occupancy, reservation trends, and traffic congestion statistics.

Class Diagram



5.4 MS4 — ECOCITY & ENVIRONMENTAL MONITORING

Business Responsibility

This microservice monitors environmental conditions across the city, including air pollution and noise levels.

It helps citizens understand the environment in their neighborhoods and allows municipal staff to detect pollution problems and take action.

Actors

• **Citizen:** Checks air quality, noise levels, and environmental conditions in different neighborhoods.

• **Environmental Agent:** Manages monitoring stations, sets pollution limits, and reviews alerts.

• **Municipal Administrator:** Monitors environmental statistics, trends, and reports.

Main Functionalities

1. **Monitoring Stations:** Create and manage virtual stations in different city locations.

2. **Air Quality Monitoring:** Track pollution indicators such as PM2.5 and PM10.

3. **Noise Monitoring:** Measure and display noise levels in different areas.

4. **Simulated Sensors:** Generate environmental measurements when real sensors are not available.

5. **Pollution Limits:** Define acceptable pollution and noise levels for each area or station.

6. **Environmental Alerts:** Automatically generate alerts when pollution exceeds the allowed limits.

7. **Environmental History:** View previous measurements and see how conditions change over time.

8. **Neighborhood Comparison:** Compare air quality and noise levels between different areas.

9. **Environmental Dashboard:** Display pollution averages, highest values, and trends through charts.

10. **Environmental Reports:** Generate reports for selected neighborhoods and periods.

5.5 MS5 — DIGITAL MUNICIPAL SERVICES

Business Responsibility

This microservice allows citizens to access municipal administrative services online instead of visiting municipal offices for every procedure.

Citizens can submit requests, upload documents, book appointments, and follow the progress of their applications.

Actors

• **Citizen:** Browses services, submits requests, uploads documents, books appointments, and checks application status.

• **Municipal Employee:** Reviews applications, checks documents, requests missing information, and approves or rejects requests.

• **Municipal Administrator:** Manages available services, sets requirements, and monitors administrative performance.

Main Functionalities

1. **Service Catalog:** Display available municipal services, their requirements, and necessary documents.

2. **Online Applications:** Allow citizens to submit administrative requests online.

3. **Document Upload:** Attach supporting documents to an application.

4. **Document Verification:** Check whether all required documents have been provided.

5. **Application Processing:** Allow employees to approve, reject, or request additional documents.

6. **Application Tracking:** Let citizens follow the status of their requests.

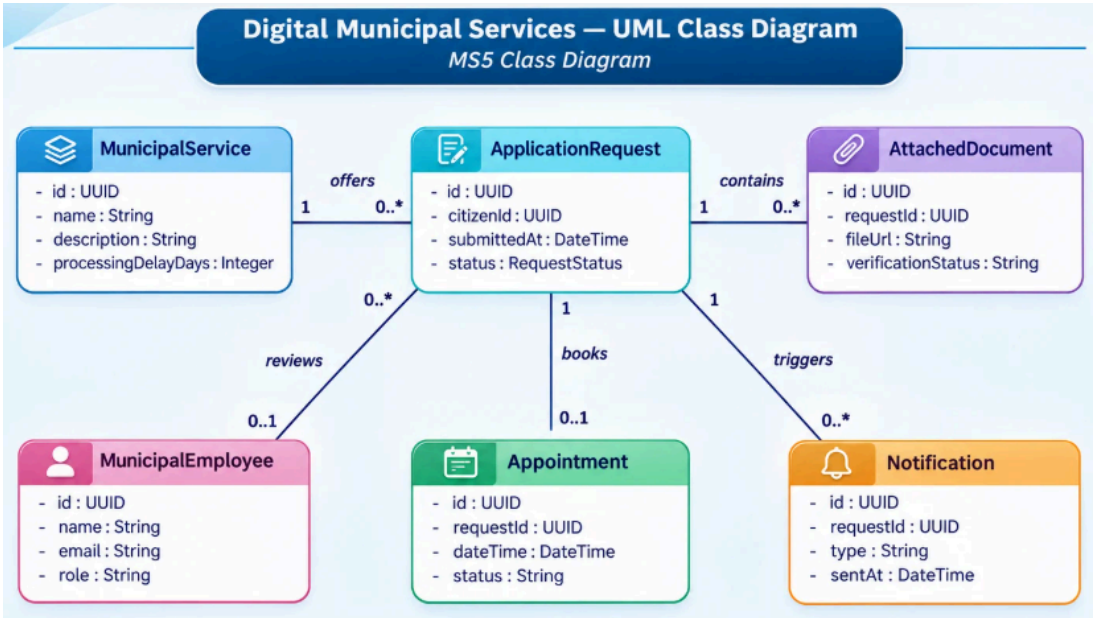
7. **Appointment Booking:** Schedule visits to municipal offices when necessary.

8. **Notifications:** Inform citizens about updates, decisions, and appointments.

9. **Application History:** Allow citizens to view their previous requests.

10. **Administrative Statistics:** Display the number of requests, average processing time, and approval rates.

Class Diagram



5.6 MS6

—

SMART PLACES & PUBLIC FACILITIES

Business Responsibility

This microservice helps citizens discover and use public facilities such as libraries, sports fields, and community centers.

It also allows citizens to reserve facilities, join community activities, and share their experiences through reviews.

Municipal staff can manage facilities, reservations, and activities through the same service.

Database: PostgreSQL (smart_places_db).

Actors

• **Citizen:** Explores facilities, checks availability, makes or cancels reservations, joins activities, and leaves reviews.

• **Facility Manager:** Manages facility information, available time slots, and booking requests.

• **Activity Organizer:** Creates community events, manages schedules, and follows participant registrations.

• **Municipal Administrator:** Supervises facilities, assigns managers, monitors bookings, and checks statistics.

Main Functionalities

1. **Facility Management:** Add, update, view, or deactivate public facilities.

2. **Facility Information:** Display location, description, opening hours, capacity, and type of facility.

3. **Availability Management:** Manage available time slots and block unavailable periods.

4. **Facility Reservations:** Allow citizens to book available facilities.

5. **Booking Approval:** Allow facility managers to accept or reject reservations when approval is needed.

6. **Reservation Cancellation:** Let citizens cancel bookings according to cancellation rules.

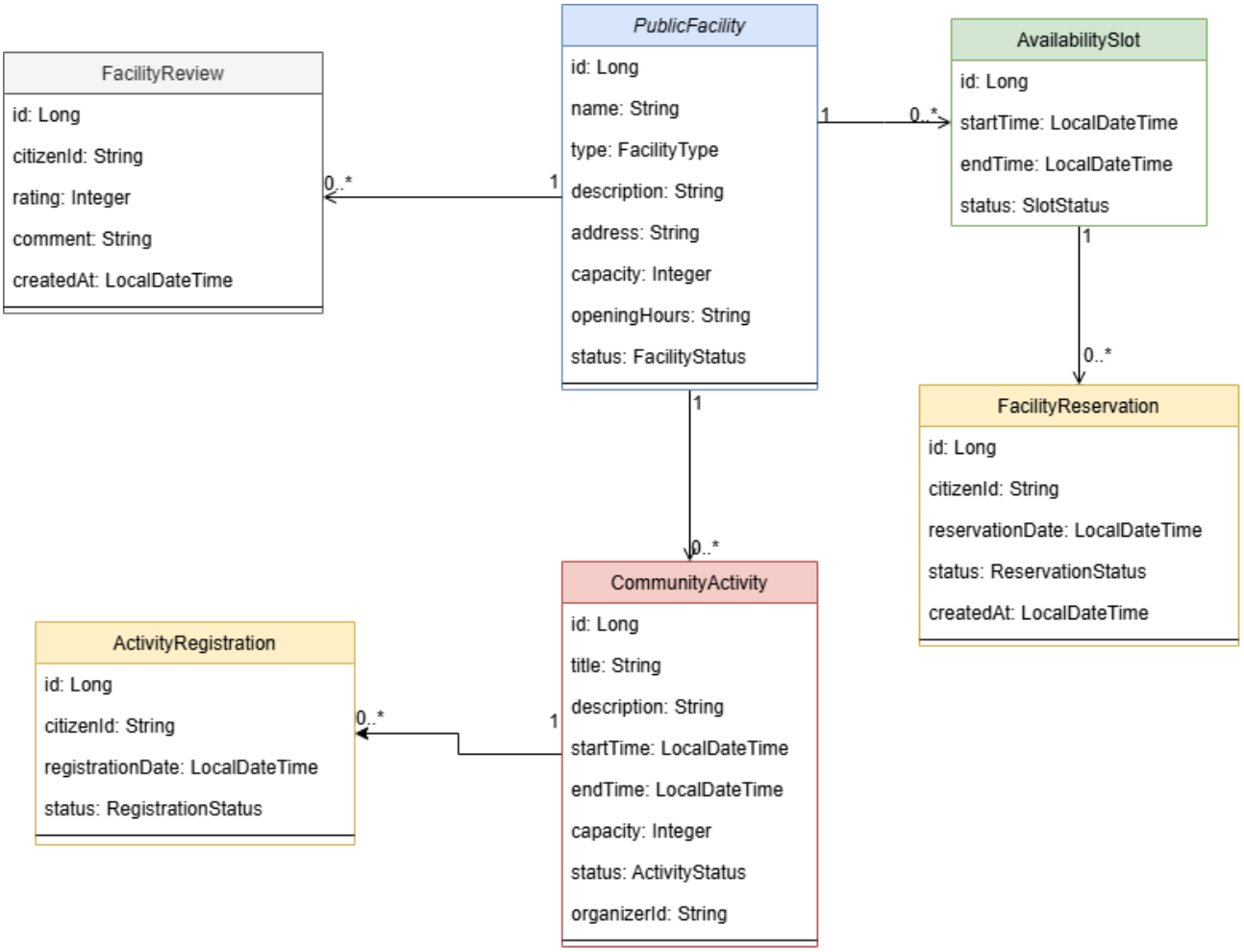
7. **Community Activities:** Create, update, publish, and cancel workshops, events, and local activities.

8. **Activity Registration:** Allow citizens to register for activities while respecting the maximum number of participants.

9. **Facility Reviews:** Let citizens rate facilities and leave comments after using them.

10. **Facility Statistics:** Display occupancy rates, popular facilities, reservation numbers, and activity participation.

Class Diagram



5.7 MS7 — RELINK AI: INTELLIGENT LOST & FOUND

Business Responsibility

ReLink AI helps residents recover lost items and pets. When someone files a lost or found report, the service compares its photos and description with existing reports and flags possible matches.

Technology

Python and FastAPI expose the API. Image and text similarity rank possible matches in the background. PostgreSQL stores reports, matches, claims, and returns. It supports keeping claim decisions and return status consistent. Photos are stored separately and referenced from the report.

Actors

• **Citizen:** Reports a lost or found item or pet, views possible matches, receives notifications, and submits proof of ownership.

• **Lost-and-found agent:** Reviews ownership claims, coordinates returns, and monitors recovery statistics.

Main Functionalities

1. **Lost and found reports:** Submit a photo, description, and approximate location.

2. **Smart matching:** Compare photos and descriptions from lost and found reports and show a similarity score.

3. **Notifications:** Alert residents when a possible match is found.

4. **Ownership verification:** Review supporting evidence before releasing an item or pet.

5. **Return management:** Arrange the handover and track its status.

6. **Recovery statistics:** Track successful returns and the time taken to complete them.

Data Model

A Report describes a lost or found item or pet and references the reporting citizen. A Match links one lost report to one found report and records its similarity score. An OwnershipClaim records the evidence and review decision. A Return records the status of the handover after approval.

Example

A student reports a lost bag. Another resident posts a photo of a bag they found nearby. ReLink AI flags the two reports as a possible match even if their descriptions use different words. The student verifies ownership before the bag is returned.

6. DATA OWNERSHIP AND DATABASE STRATEGY

Each microservice owns its database. No service reads or writes another service's tables directly. A service may store an external identifier such as citizenId or sourceIncidentId, but that identifier is not a cross-database foreign key.

Proposed database allocation

SERVICE	DATABASE	DATA OWNED BY THIS SERVICE
MS1 · Citizen & Civic	PostgreSQL	Citizen profiles, proposals, consultations, votes and comments.
MS2 · Incident & Maintenance	PostgreSQL	Incidents, technicians, work orders, interventions, resolution confirmations and notifications.
MS3 · Mobility & Traffic	PostgreSQL	Road segments, closures, traffic conditions, parking areas, spaces and reservations.
MS4 · EcoCity	PostgreSQL	Monitoring stations, environmental readings, thresholds and pollution alerts.
MS5 · Digital Municipal Services	PostgreSQL	Service catalog, applications, uploaded-document metadata, appointments and request history.
MS6 · Smart Places	PostgreSQL	Facilities, time slots, reservations, activities, registrations and reviews.
MS7 · ReLink AI	PostgreSQL	Lost and found reports, photo references, match scores, ownership claims, return records, and match notifications.

Why PostgreSQL

• Relational entities and clear foreign-key relationships within each service.

• Transactions and constraints are useful for votes, reservations, applications and work orders.

• SQL supports filtering, reporting, aggregation and operational dashboards.

• PostGIS can be considered later for advanced geographic queries if required.

7. INTER-MICROSERVICE COMMUNICATION

Communication principles

• **Synchronous REST/OpenFeign:** use when the caller needs an immediate response, such as checking a resource or requesting a calculation.

• **Asynchronous messaging:** use when a business event can be processed later, such as notifying another service that a road closure was created.

• Each service remains available independently as far as possible. A temporary failure in a downstream service should not erase a successfully stored local transaction.

Proposed service interactions

FLOW	WHAT IS EXCHANGED	COMMUNICATION RULE
MS1 → other services	Citizen ID and, only when needed, a small profile summary.	Synchronous lookup on demand; consumers keep citizenId as an external reference.
MS2 → MS3	RoadDisruptionDeclared: event ID, incident ID, affected location/road segment, severity and timestamp.	Asynchronous event; MS3 updates its mobility view without accessing MS2's database.
MS3 → MS2 (optional)	Reference to a newly discovered road issue or mobility impact.	Optional event or link to an existing report; do not create duplicate incident records.
MS4 → web notification layer (optional)	Environmental alert type, affected zone, measured value and threshold.	MS4 remains independently operable; publish an alert only if this integration is implemented.
MS5 → identity context	Authenticated citizen ID and role claims from the access token.	Use token claims and an external citizenId; no direct database access to MS1.
MS6 → identity context	Authenticated citizen ID for a facility reservation or activity registration.	Use token claims and an external citizenId; MS6 owns its bookings.

MS7 → background matching worker	Report ID and report type. MS7 identifies the reporting user from the Keycloak access token.	Use the token for identity. Run matching asynchronously so reports can be saved even when the worker is unavailable.

8. GLOBAL TECHNICAL ARCHITECTURE

Web stack

• **Frontend:** Angular web application.

• **API Gateway:** single entry point for frontend requests and routing to backend services.

• **Eureka Discovery Server:** service registration and discovery.

• **Spring Cloud Config Server:** centralized configuration for Spring Boot services.

• **Keycloak:** authentication and role-based access management.

• **Business microservices MS1–MS6:** Spring Boot services, each with its own PostgreSQL database.

• **Advanced microservice MS7:** ReLink AI, built with Python and FastAPI, with its own PostgreSQL database and a background matching worker.

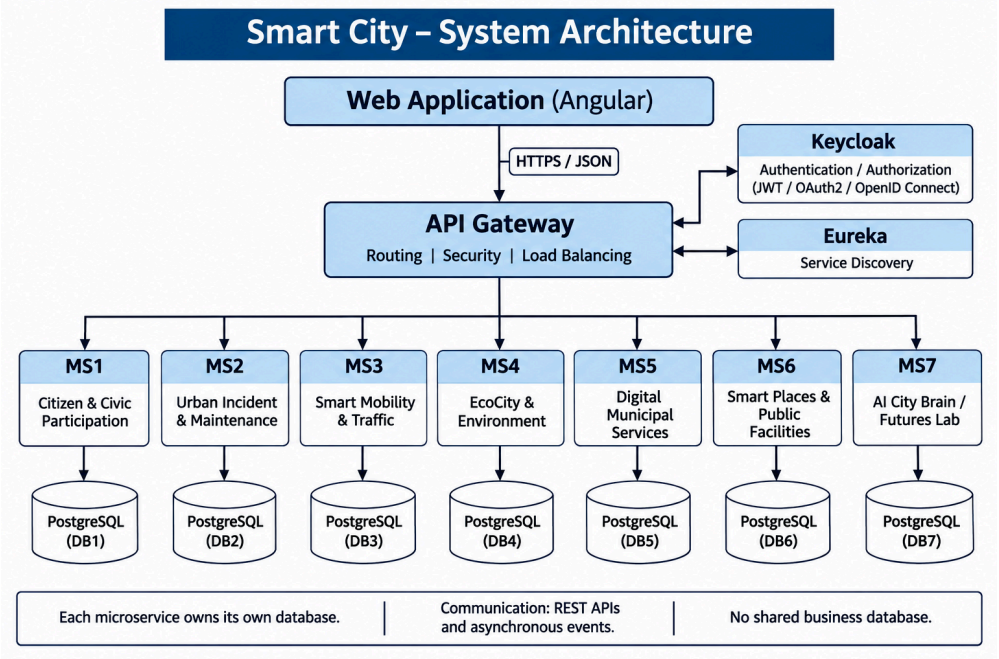
• **Inter-service synchronous communication:** REST/OpenFeign where appropriate.

• **Asynchronous communication:** RabbitMQ (or the messaging technology approved by the instructor).

• **Docker:** containerization of the application components.

• **Monitoring/logging:** centralized logs and health checks as the project grows

Architecture flow



9. GLOBAL USE CASE DIAGRAM — ACTOR AND USE CASE INVENTORY

The global use case diagram should show the Smart City system boundary, the actors outside it and grouped use cases inside it. Avoid drawing database entities in a use case diagram.

Actors

• Citizen.

• Municipal administrator.

• Municipal agent/service officer.

• Technician.

• Mobility operator.

• Facility manager/activity organizer.

• Lost-and-found agent.

• System administrator.

Use cases grouped by module

MS1: Manage profile; manage neighborhood; submit proposal; comment; participate in consultation; vote; view participation history.

MS2: Report incident; upload evidence; track incident; validate incident; request information; categorize and prioritize; create work order; assign technician; schedule intervention; update progress; verify resolution; confirm resolution; view incident statistics.

MS3: View traffic; manage road closure; manage parking facility and spaces; view availability; reserve/cancel parking; consult parking history; view alternative route; view mobility statistics.

MS4: Manage monitoring station; generate/record reading; configure threshold; view environmental history; compare neighborhoods; view pollution alert; consult environmental dashboard; generate report.

MS5: Browse service catalog; submit request; upload documents; validate application; request missing documents; approve/reject; book appointment; track request; view request history; consult administrative statistics.

MS6: Browse facilities; view availability; reserve/cancel facility; validate booking; publish activity; register for activity; review facility; view facility statistics.

MS7: Report a lost or found item or pet; view possible matches; receive match notifications; submit proof of ownership; verify a claim; coordinate and track a return; view recovery statistics.

System administrator: manage technical configuration and monitor service health.

11. REST API DESIGN AND HTTP RESPONSE CONVENTIONS

MS1 — Citizen & Civic Participation

• POST /api/citizens — create profile (201 Created).

• GET /api/citizens/{id} — retrieve profile (200 OK, 404 Not Found).

• PUT /api/citizens/{id} — update profile (200 OK).

• POST /api/proposals — submit proposal (201 Created).

• GET /api/proposals/{id} — retrieve proposal (200 OK).

• POST /api/consultations — publish consultation (201 Created).

• POST /api/consultations/{id}/votes — cast vote (201 Created; 409 Conflict for duplicate vote).

• POST /api/proposals/{id}/comments — add comment (201 Created).

MS2 — Urban Incident & Maintenance

• POST /api/incidents — create incident (201 Created).

• GET /api/incidents — list/filter incidents (200 OK).

• GET /api/incidents/{id} — retrieve incident (200 OK, 404 Not Found).

• PATCH /api/incidents/{id}/validation — accept/reject/request information (200 OK, 409 for invalid transition).

• PATCH /api/incidents/{id}/priority — set confirmed priority (200 OK).

• POST /api/incidents/{id}/work-orders — create work order (201 Created).

• POST /api/work-orders/{id}/assign — assign technician (200 OK).

• PATCH /api/work-orders/{id}/schedule — schedule work (200 OK, 409 for overlapping schedule).

• PATCH /api/work-orders/{id}/status — update progress (200 OK, 409 for invalid transition).

• POST /api/incidents/{id}/confirmations — submit citizen confirmation (201 Created).

• GET /api/incidents/statistics — retrieve KPIs (200 OK).

MS3 — Smart Mobility & Traffic

• GET /api/traffic/conditions — view traffic conditions (200 OK).

• POST /api/road-closures — register closure (201 Created).

• GET /api/parking-facilities — list facilities (200 OK).

• GET /api/parking-spaces/availability — check spaces (200 OK).

• POST /api/parking-reservations — reserve space (201 Created; 409 if unavailable).

• PATCH /api/parking-reservations/{id}/cancel — cancel reservation (200 OK).

• GET /api/parking-reservations/history — reservation history (200 OK).

MS4 — EcoCity

• POST /api/monitoring-stations — create station (201 Created).

• POST /api/readings — record/simulate reading (201 Created).

• GET /api/readings — filter by station, metric and period (200 OK).

• POST /api/thresholds — configure threshold (201 Created).

• GET /api/pollution-alerts — list alerts (200 OK).

• GET /api/environmental-reports — generate/read report (200 OK).

MS5 — Digital Municipal Services

• GET /api/municipal-services — list catalog (200 OK).

• POST /api/service-requests — submit application (201 Created).

• POST /api/service-requests/{id}/documents — upload document metadata (201 Created).

• PATCH /api/service-requests/{id}/decision — approve/reject/request documents (200 OK).

• POST /api/service-requests/{id}/appointments — book appointment (201 Created).

• GET /api/service-requests/{id} — track request (200 OK).

MS6 — Smart Places

• POST /api/facilities — register facility (201 Created).

• GET /api/facilities — browse facilities (200 OK).

• GET /api/facilities/{id}/availability — view slots (200 OK).

• POST /api/facility-reservations — book slot (201 Created; 409 if unavailable).

• POST /api/activities — publish activity (201 Created).

• POST /api/activities/{id}/registrations — register citizen (201 Created; 409 if full).

• POST /api/facilities/{id}/reviews — submit review (201 Created).

MS7 — ReLink AI

• POST /api/lost-found/reports — file a lost or found report (201 Created).

• GET /api/lost-found/reports/{id}/matches — view candidate matches and scores (200 OK, 404 Not Found).

• POST /api/lost-found/matches/{id}/claims — submit proof of ownership (201 Created).

• PATCH /api/lost-found/claims/{id} — approve or reject a claim (200 OK, 403 Forbidden, 409 Conflict).

• PATCH /api/lost-found/returns/{id} — update the handover status (200 OK, 409 Conflict).

• GET /api/lost-found/statistics — retrieve recovery and return statistics (200 OK, 403 Forbidden).

17. CONCLUSION

Smart City is designed as a modular web platform organized around municipal business capabilities. Each microservice owns its data and exposes well-defined APIs or events. The separation allows teams to develop services independently while preserving controlled collaboration across the platform.

Each microservice owns a municipal workflow and its data. ReLink AI adds lost-and-found matching to the platform. It suggests possible matches using photos and descriptions, while ownership is verified before a return.